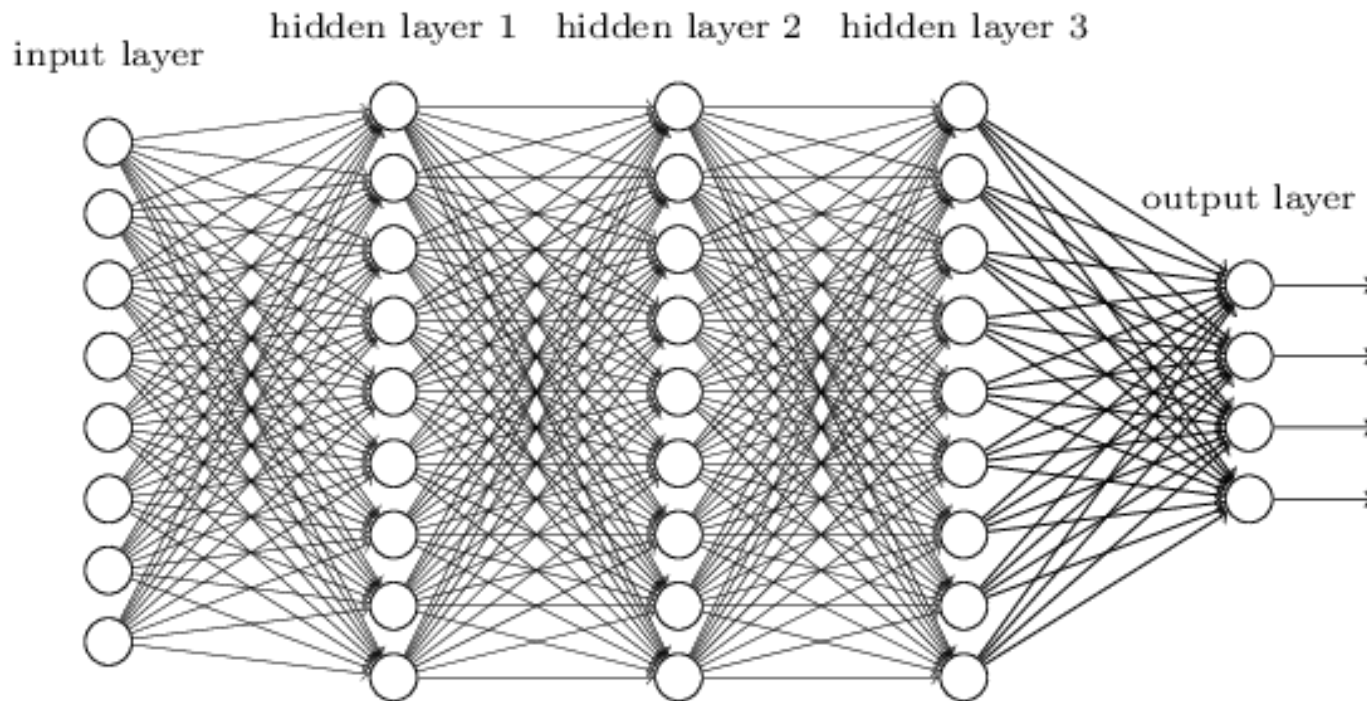

Lecture 9

Convolution Neural Network

Smaller Network: ANN



Need for CNN

According to a MathWork post, a CNN convolves learned **features with input data**, and uses 2D convolutional layers, making this architecture well suited to processing 2D data, such as images.

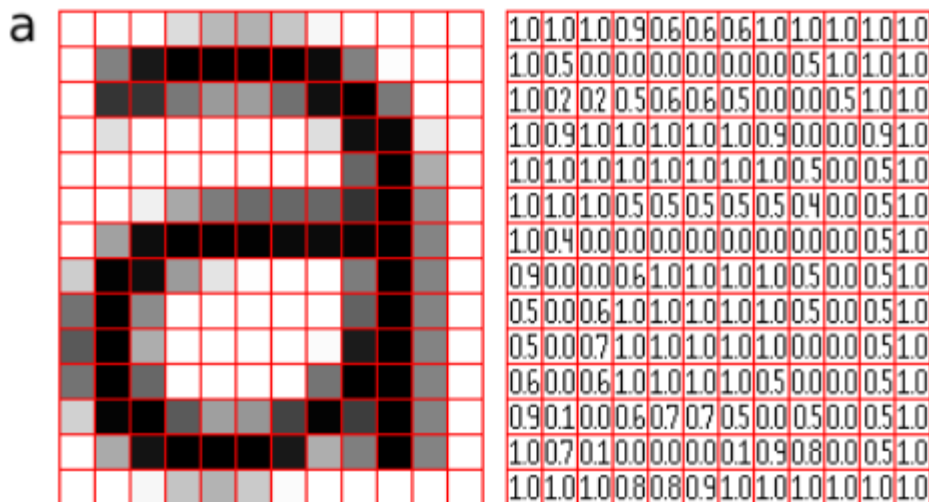
Since CNNs eliminate the need for manual feature extraction, one doesn't need to select features required to classify the images.

Convolutional Neural Network

A Convolutional Neural Network, also known as CNN or ConvNet, is a class of [neural networks](#) that specializes in processing data that has a grid-like topology, such as an image.

A digital image is a binary representation of visual data.

It contains a series of pixels arranged in a grid-like fashion that contains pixel values to denote how bright and what color each pixel should be.



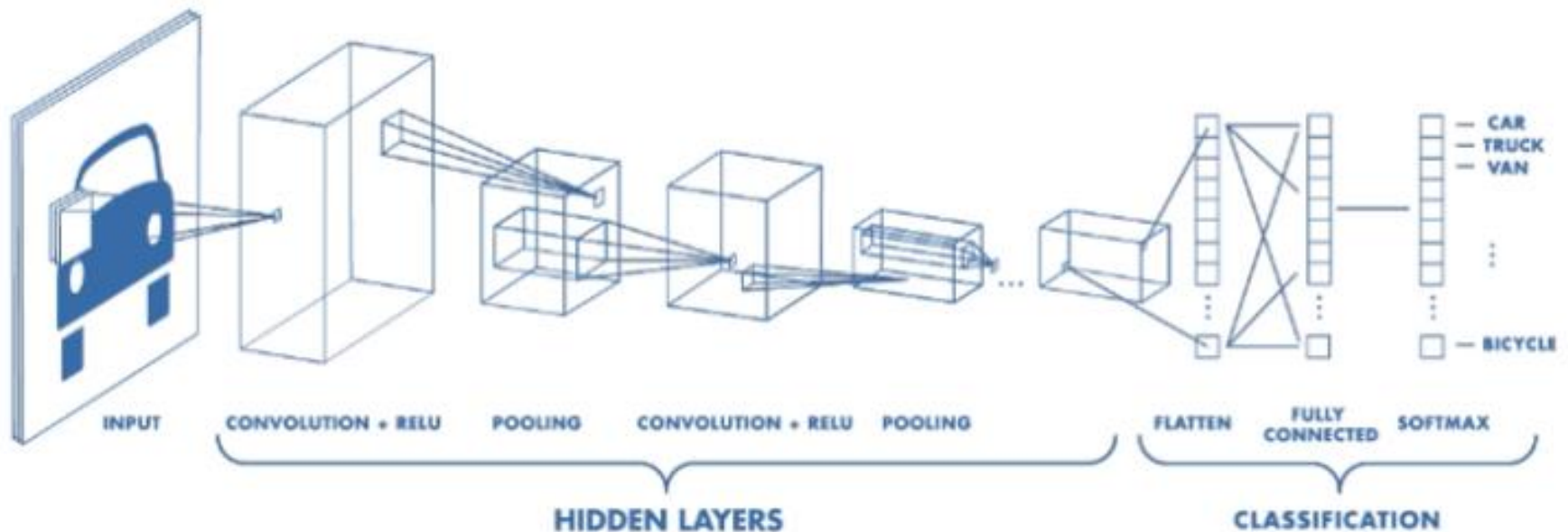
Convolutional Neural Network

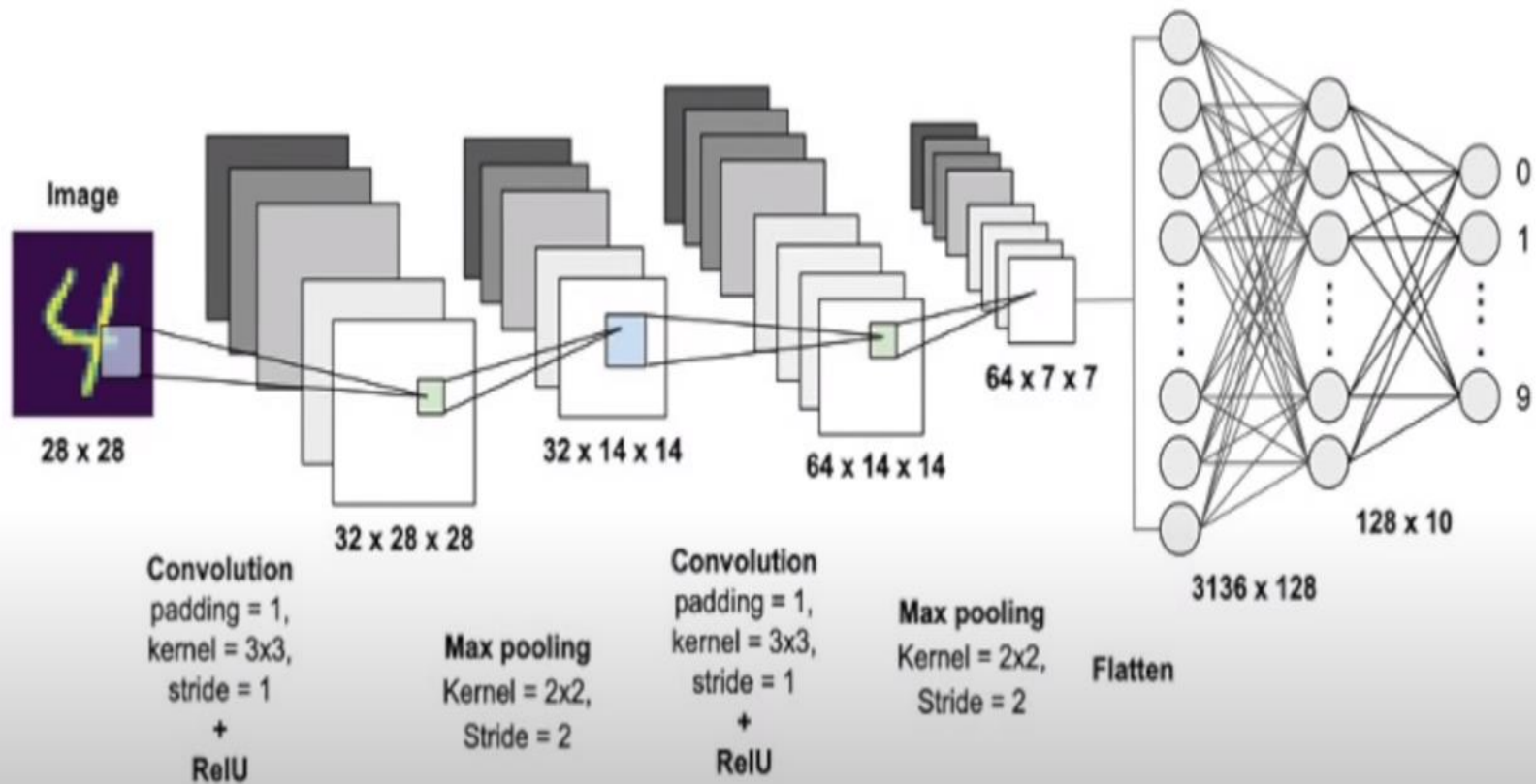
A CNN typically has three layers:

1. a convolutional layer,
2. a pooling layer,
3. a fully connected layer.

The convolution layer is the core building block of the CNN. It carries the main portion of the network's computational load.

Architecture of a CNN





Convlayer

This layer performs a dot product between two matrices, where one matrix is the set of learnable parameters known as a kernel, and the other matrix is the restricted portion of the receptive field (image or part of image).

The kernel is spatially smaller than an image but is more in-depth. This means that, if the image is composed of three (RGB) channels, the kernel height and width will be spatially small, but the depth extends up to all three channels.

Kernel

- Kernel is nothing but a filter that is used to extract the features from the images.
- the kernel is a matrix that moves over the input data, performs the dot product with the sub-region of input data, and gets the output as the matrix of dot products.
- Kernel moves on the input data by the stride value.
- Output dimension after kernel = $[i - k] + 1$

Stride

During the forward pass, the kernel slides across the height and width of the image-producing the image representation of that receptive region.

This produces a two-dimensional representation of the image known as an **activation map** that gives the response of the kernel at each spatial position of the image.

The sliding size of the kernel is called a **Stride**.

Output dimension after stride = $\lfloor (i - k) / s \rfloor + 1$

Padding

- Padding is the best approach, where the number of pixels needed for the convolutional kernel to process the edge pixels are added onto the outside copying the pixels from the edge of the image.
- Fix the border effect problem with padding.
- Output dimension after stride = $\lfloor (i - k + 2p) / s \rfloor + 1$

Pooling Layer

The pooling layer replaces the output of the network at certain locations by deriving a summary statistic of the nearby outputs.

This helps in reducing the spatial size of the representation, which decreases the required amount of computation and weights.

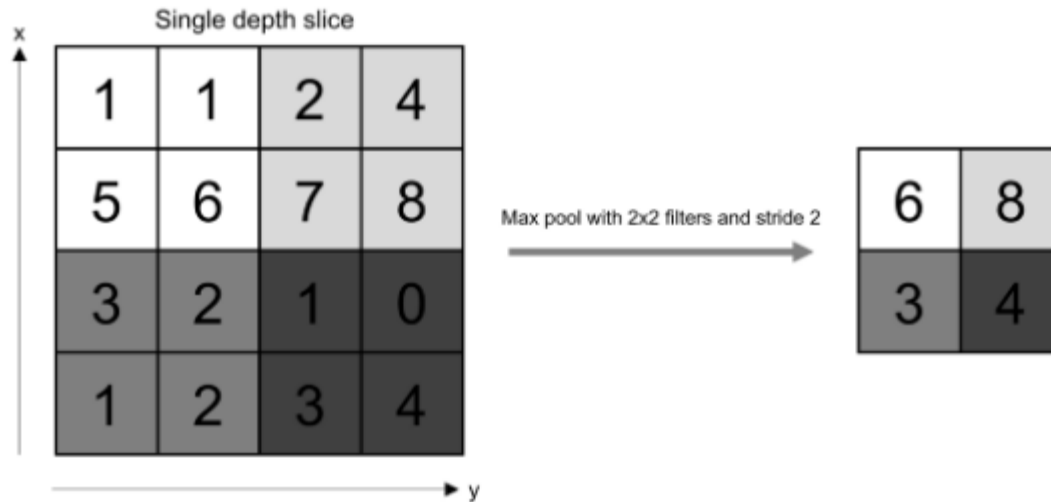
The pooling operation is processed on every slice of the representation individually.

Types Of Pooling

☐ Max Pooling

☐ Average Pooling

Max Pooling Operation



Flattening

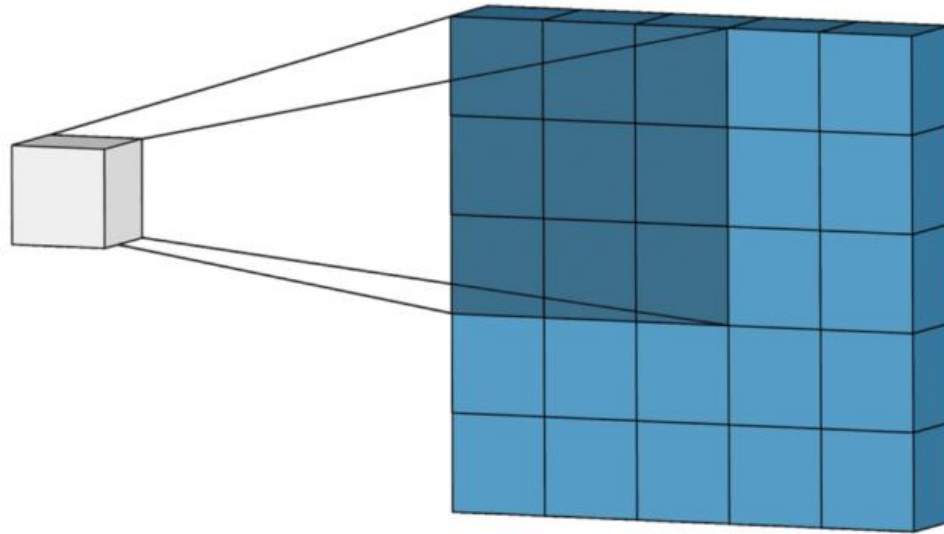
Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector. The flattened matrix is fed as input to the fully connected layer to classify the image.

Fully Connected Layer

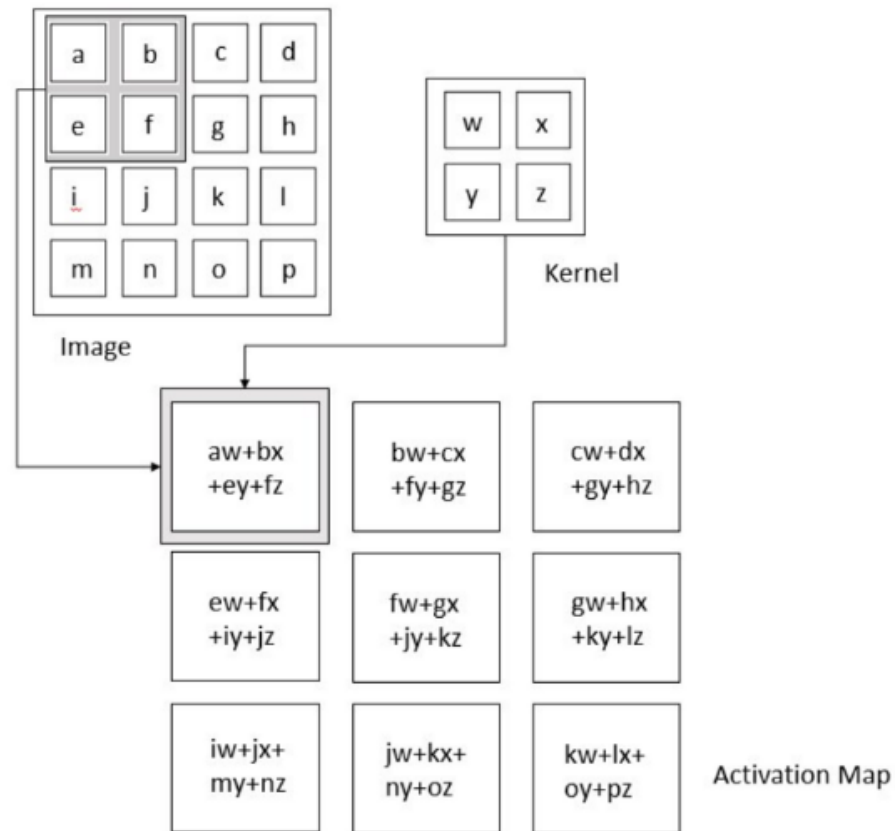
Neurons in this layer have full connectivity with all neurons in the preceding and succeeding layer as seen in regular FCNN. This is why it can be computed as usual by a matrix multiplication followed by a bias effect.

The FC layer helps to map the representation between the input and the output.

Illustration of Convolution Operation



Activation Map



Consider learning an image:

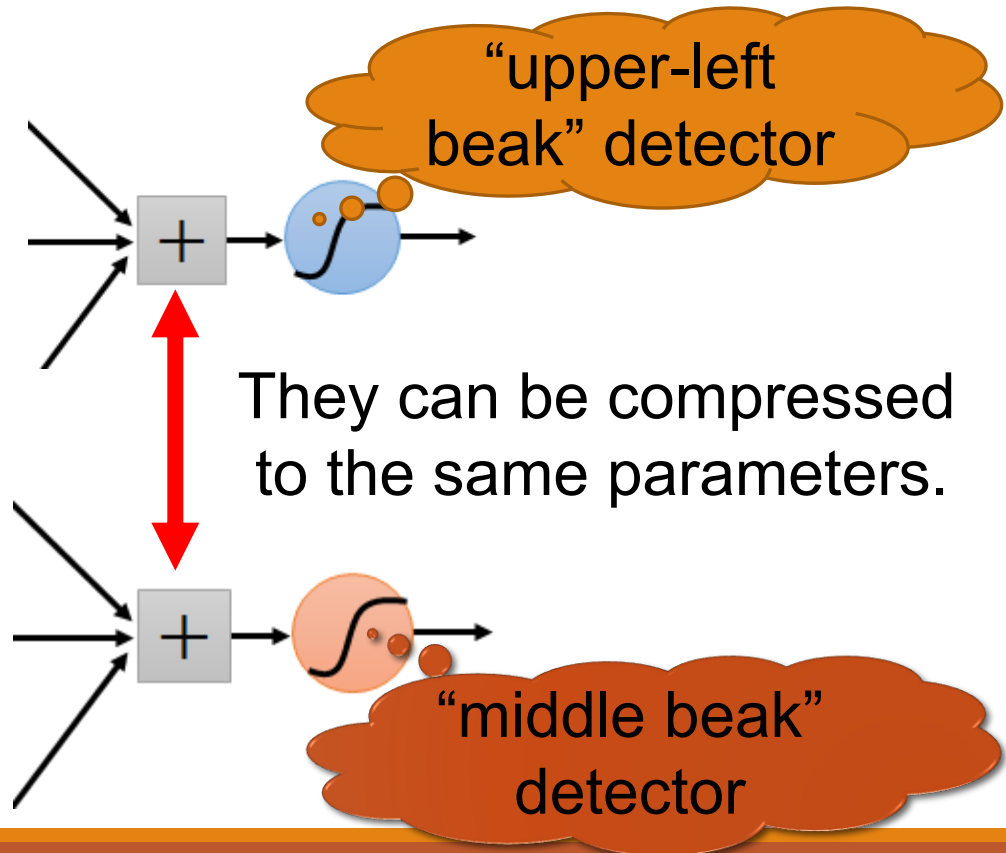
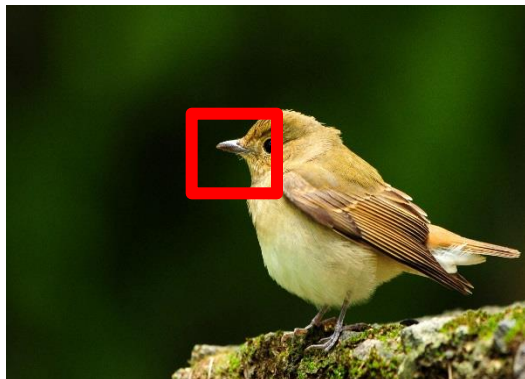
Some patterns are much smaller than the whole image

Can represent a small region with fewer parameters



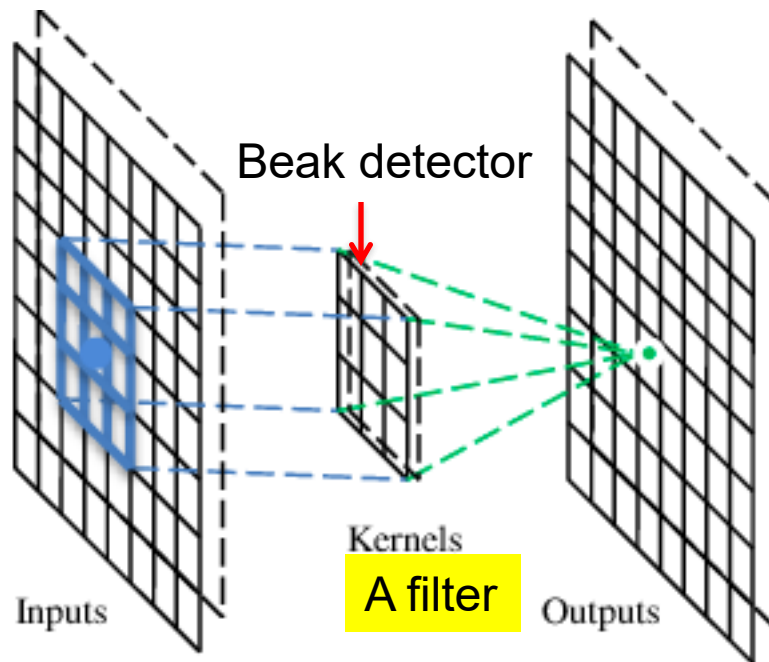
Same pattern appears in different places:
They can be compressed!

What about training a lot of such “small” detectors
and each detector must “move around”.



A convolutional layer

A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.



Convolution

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

Convolution

stride=1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot
product



3

-1

6 x 6 image

Convolution

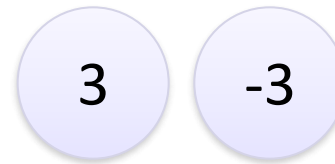
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



Convolution

stride=1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Convolution

stride=1

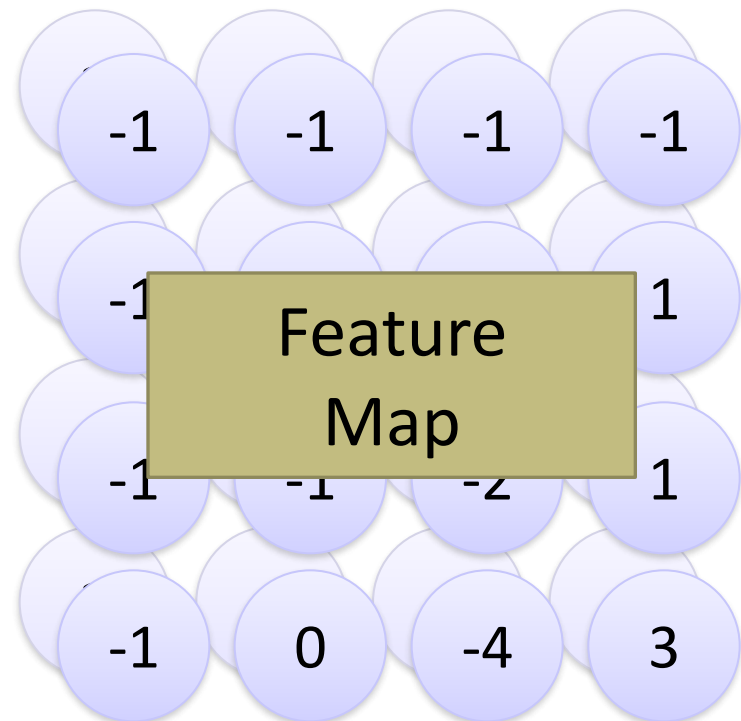
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Repeat this for each filter



Two 4 x 4 images
Forming 2 x 4 x 4 matrix

Color image: RGB 3 channels

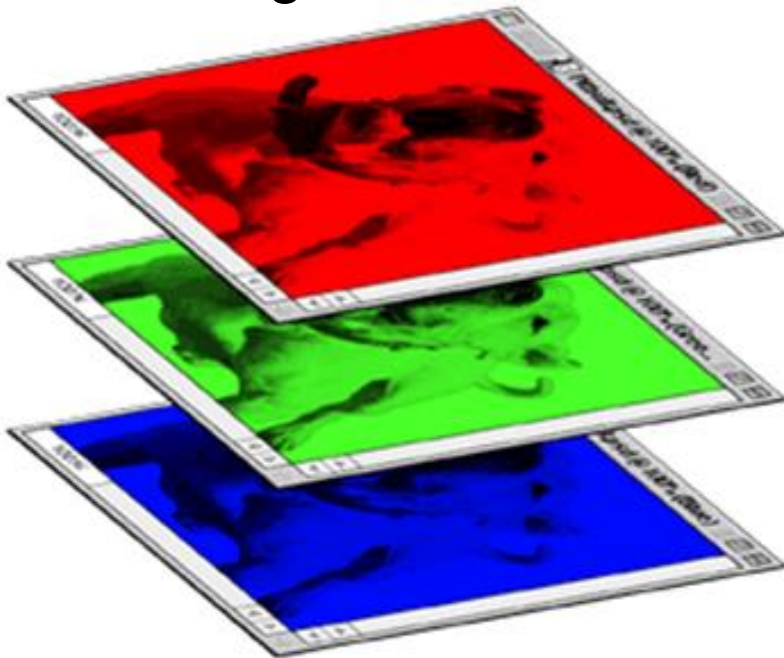
		1	-1	-1
		-1	1	-1
		-1	-1	1

Filter 1

		-1	1	-1
		-1	1	-1
		-1	1	-1

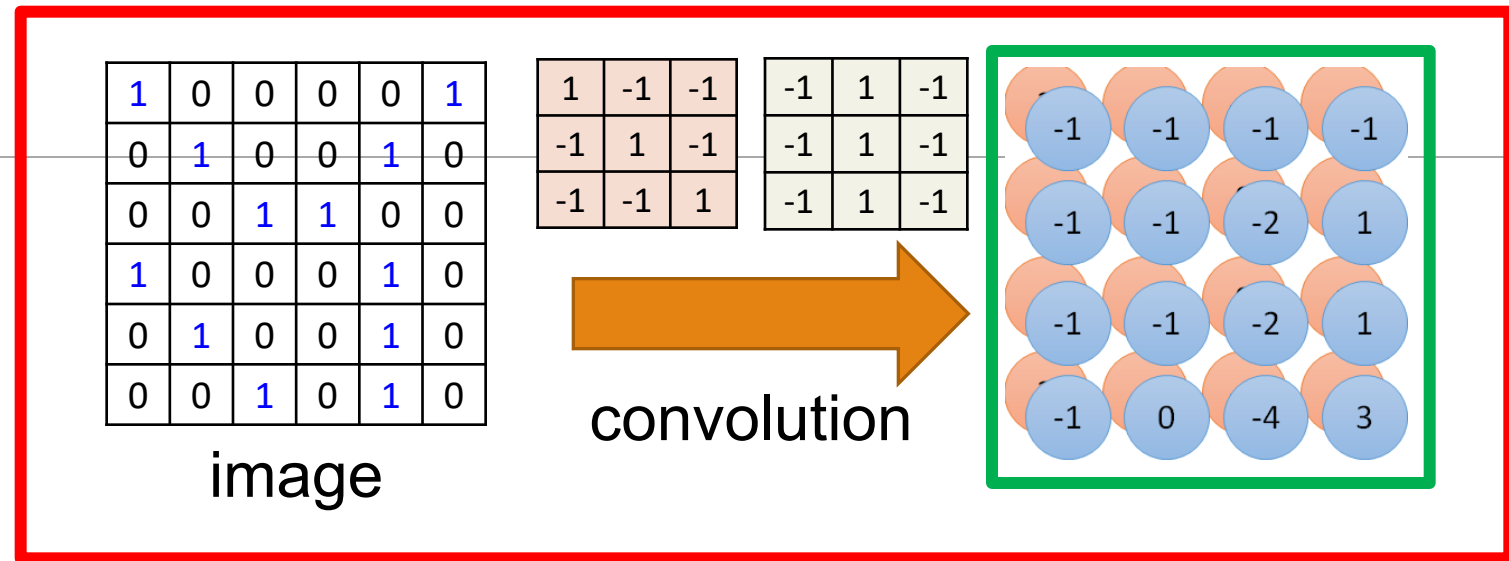
Filter 2

Color image



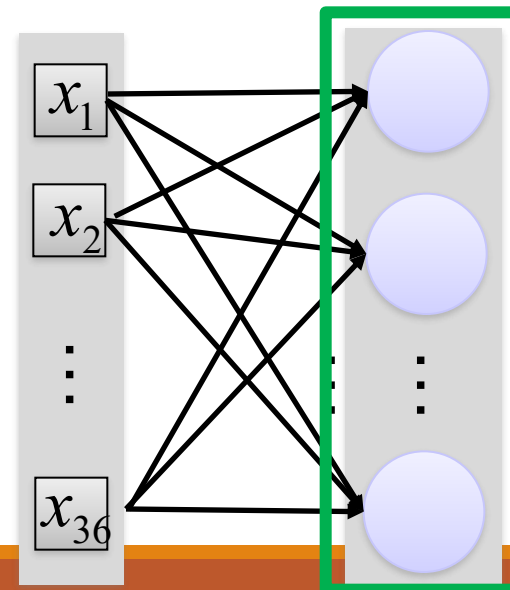
	1	0	0	0	0	1
	0	1	0	0	1	0
	0	0	1	1	0	0
	1	0	0	0	1	0
	0	1	0	0	1	0
	0	0	1	0	1	0

Convolution v.s. Fully Connected



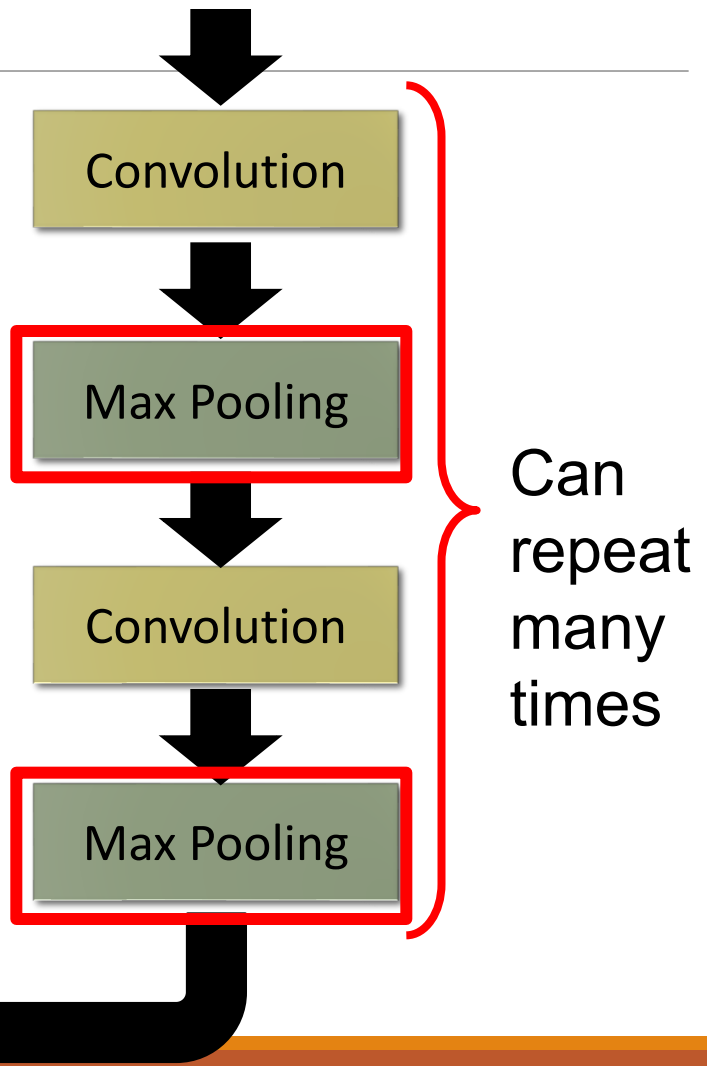
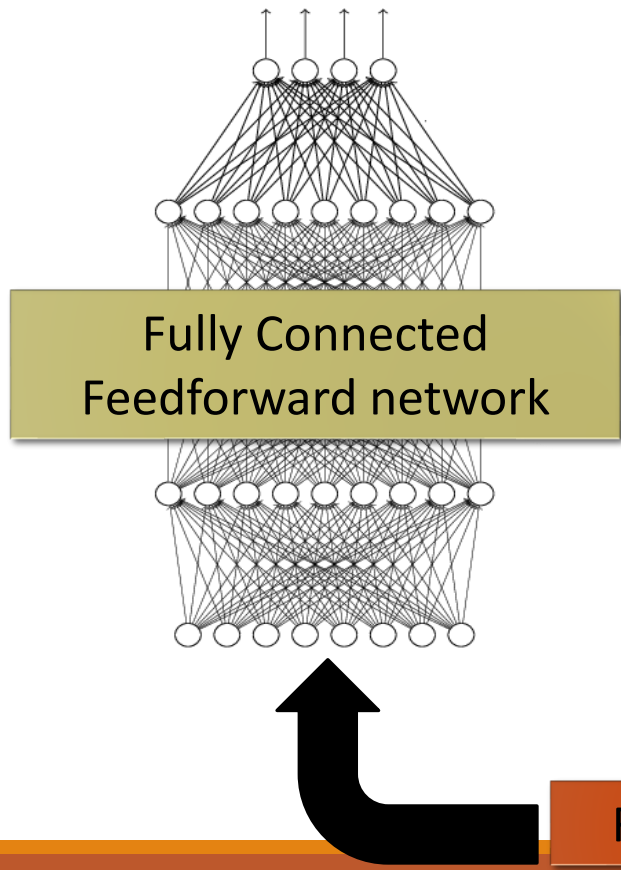
Fully-
connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



The whole CNN

cat dog



Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Why Pooling

Subsampling pixels will not change the object

bird



Subsampling

bird



We can subsample the pixels to make image



fewer parameters to characterize the image

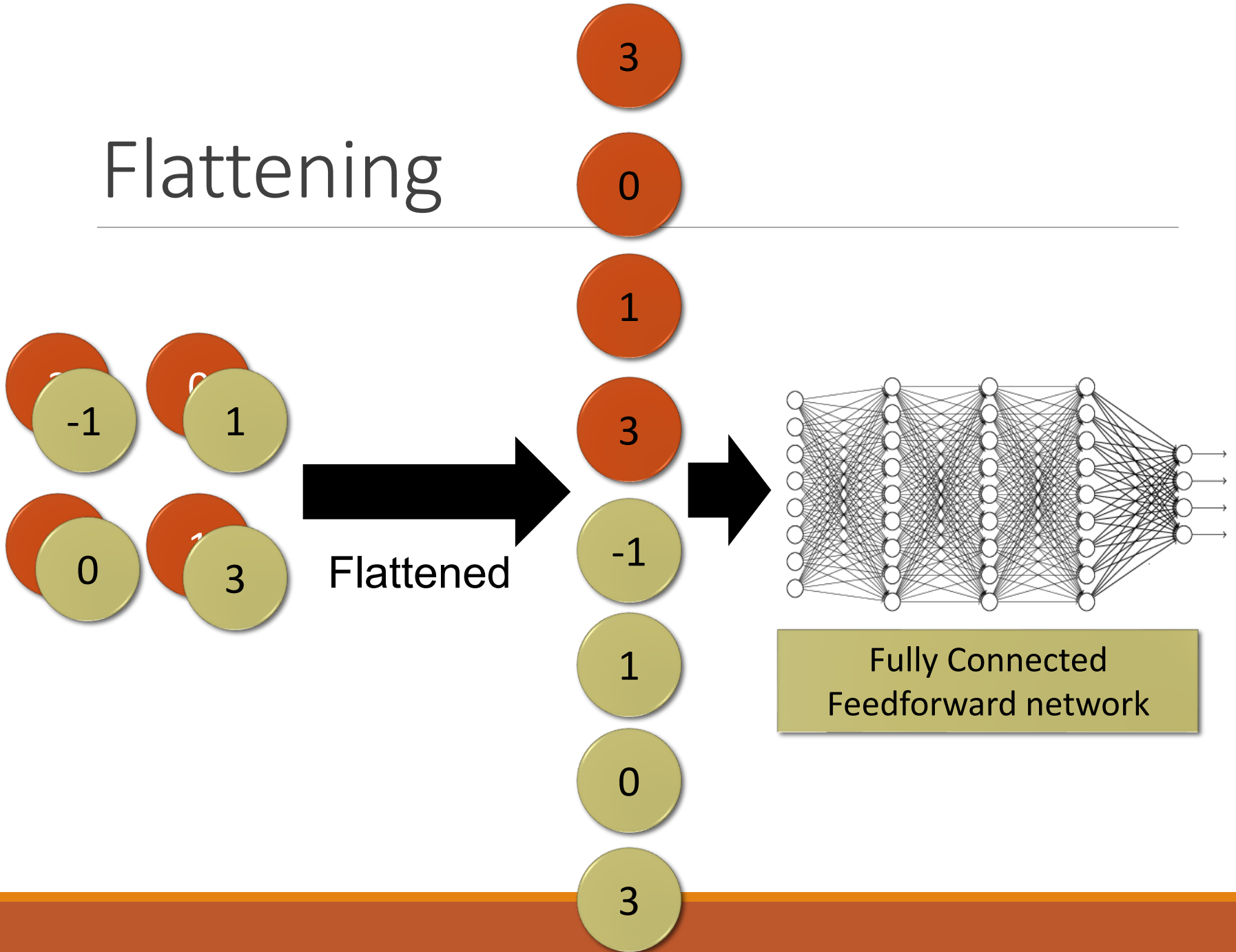
A CNN compresses a fully connected network in two ways:

Reducing number of connections

Shared weights on the edges

Max pooling further reduces the complexity

Flattening



CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D tensor)

```
model2.add( Convolution2D( 25, 3, 3,  
                           input_shape=(28, 28, 1)) )
```

1	-1	-1
-1	1	-1
-1	-1	-1

-1	1	-1
-1	1	-1
-1	1	-1

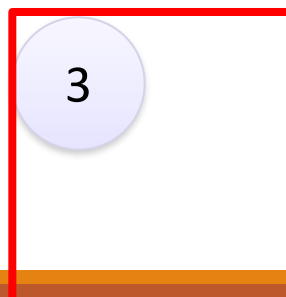
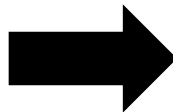
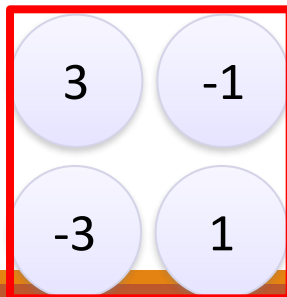
There are
25 3x3
filters.

Input_shape = (28 , 28 , 1)

28 x 28 pixels

1: black/white, 3: RGB

```
model2.add(MaxPooling2D( (2, 2) ))
```



input

Convolution

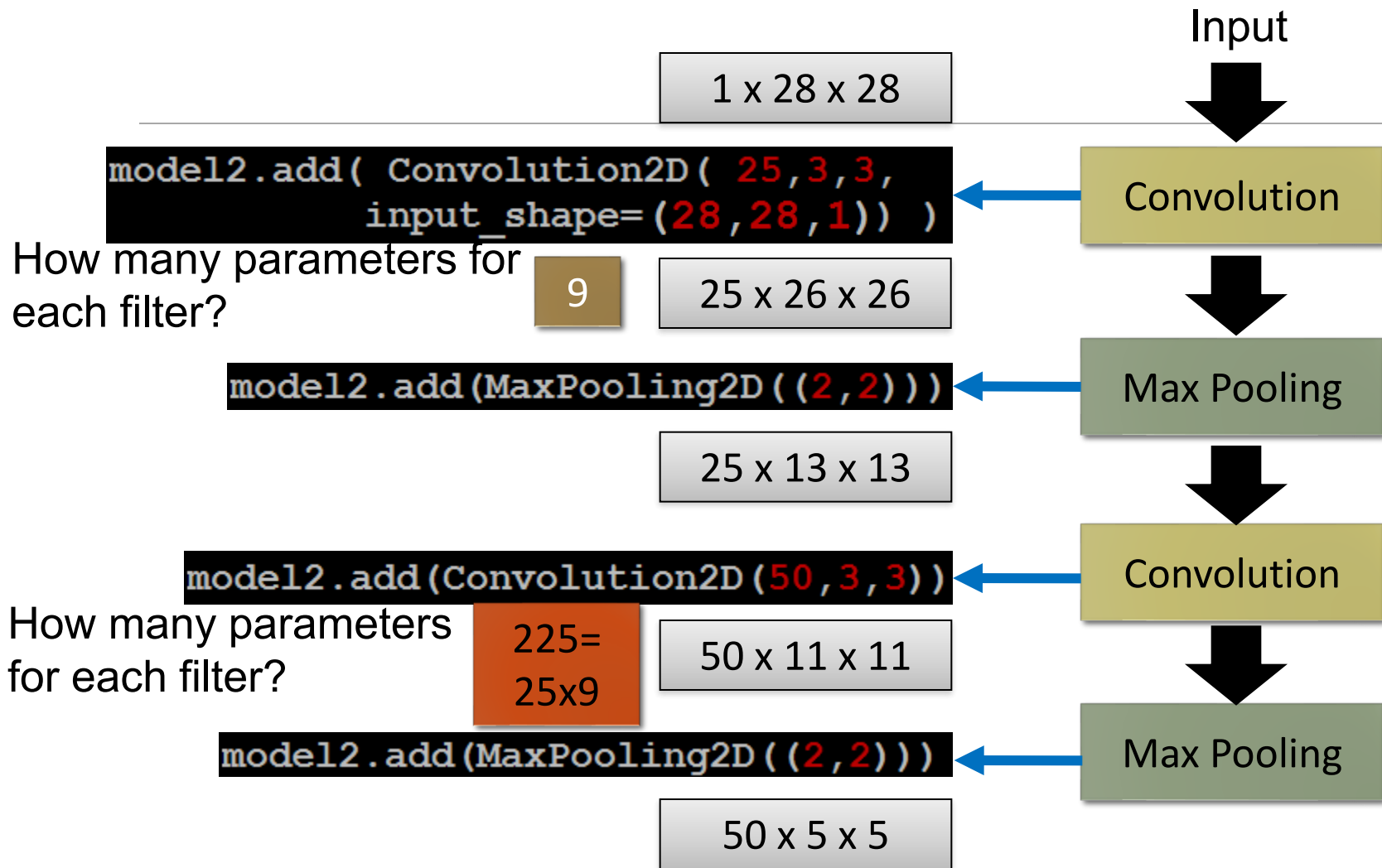
Max Pooling

Convolution

Max Pooling

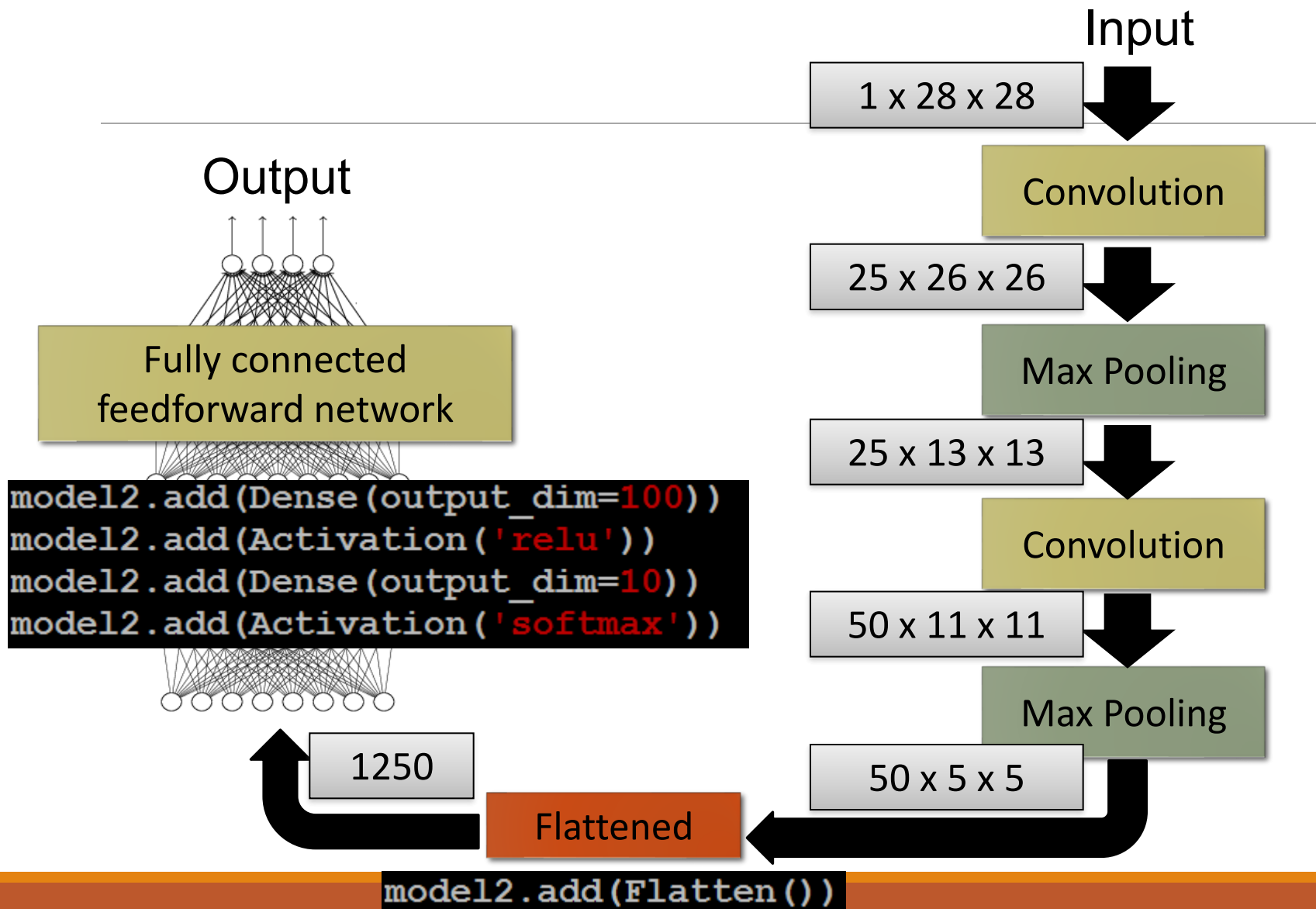
CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*



CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*



Back Propagation

- Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 0.5 and learning rate is 1. Perform another forward pass.

